

Generic antenna information extraction pipeline: from PDF to structured output

1 Background and motivation

The **antennaAuto** project currently extracts patch-antenna designs via a deterministic pipeline. The retrieval plan includes fixed queries for `classification`, `materials`, `layers`, `parameters`, `entities`, `feeds` and `quality` ¹. This means each paper is searched for phrases like “**substrate thickness**,” “**slot geometry**,” “**feed location**,” etc., regardless of whether the paper describes a microstrip patch, a horn, a helix or a reflector ¹. Candidate records are ranked based on BM25 and dense embeddings, but there is no dynamic adaptation: base queries run first and phase-one queries are always appended ¹. The canonicalisation system prompt explicitly tells the model to preserve patch geometry, slot/notch geometry, feed geometry and ground-plane geometry ². While this works for microstrip patches, it cannot discover the parameters of arbitrary antennas.

To support **arbitrary antennas**, we propose a comprehensive, open-source pipeline that classifies the antenna type, adapts retrieval to the design, uses both GROBID and NuExtract3 for precise extraction, and produces readable structured outputs at each stage.

2 Architecture overview

The pipeline is divided into five phases, with open-source models at each stage.

Phase 1: Ingestion and parsing

1. **Convert the PDF into a structured document:**
2. Use **Docling** as the primary parser to convert the raw PDF into a structured page-object model with text chunks, tables, figures and bounding boxes.
3. Use **GROBID** to extract metadata and full text into TEI/XML. GROBID is a machine-learning library that extracts and structures PDFs into XML/TEI documents; it can parse full text and provide coordinates of identified structures such as section titles, figures and tables ³. It performs header extraction, reference parsing and full text structuring ⁴, and yields bounding boxes for extracted information ⁵.
4. For scanned pages or complex mathematics, apply **MonkeyOCR v1.5** or **olmOCR-2**. These models parse mathematical formulas and multi-level tables with high accuracy.
5. **Optional visual markup extraction:**

6. Use **NuExtract3** to convert pages with complex layouts into Markdown when Docling fails.
NuExtract3 is a 4-billion-parameter vision-language model that combines structured information extraction with high-quality image-to-Markdown conversion ⁶. It accepts multimodal inputs (text, images or both) and outputs Markdown and JSON ⁷. This helps to recover table structures and figure captions lost by Docling.

7. Unified evidence store:

8. Store all parsed units (chunks, sections, tables, table rows, figures) with fields: `evidence_id`, `type` (chunk/section/table/figure), `page_number`, `text / markdown`, bounding boxes and metadata (e.g., table row context). Each item is uniquely addressable and can be retrieved later.

Phase 2: Classification and query planning

1. **Abstract extraction:** isolate the abstract and introduction using GROBID's section tags (e.g., `<abstract>`, `<sec>`).
2. **Antenna classification:** call an open-source LLM such as **Gemma 4** (E4B or 26B variant) or **LLaMA-3** to read the abstract and assign:
3. **Antenna class:** microstrip/patch, array, horn, helix, spiral, slot, loop, wire, reflector, metamaterial or other.
4. **Radiating element shapes:** rectangular, circular, elliptical, triangular, tapered slot, conical, helical, etc.
5. **Feed type:** coaxial probe, microstrip line, waveguide, aperture coupling, microcoax, etc.
6. **Substrate presence** and number of layers.
7. **Performance metrics:** resonant frequency, bandwidth, gain, directivity, polarization.

This classification guides retrieval and ensures that parameters relevant to horns, helices or reflectors are not missed.

1. **Dynamic query generation:** request the same LLM to generate a set of search queries tailored to the antenna class and each schema slot. For example, for a helical antenna the LLM might produce queries like `"helix diameter pitch"`, `"number of turns helical antenna"`, `"ground plane disc radius"` and `"feeding method coaxial probe location"`. Each query is stored as a **SearchQuery** object with fields: `query_id`, `query_text`, `priority` and `why`. These queries replace the static lists used in the current pipeline ¹.
2. **InterpretationRecord:** save the classification results, generated queries and open uncertainties as an intermediate JSON file. Example structure:

```
{
  "antenna_class": "helical",
  "radiating_elements": ["helix"],
  "feed_type": "coaxial probe",
```

```

"substrate_present": false,
"search_queries": [...],
"open_uncertainties": ["number of layers unknown"]
}

```

This early output allows inspection and correction before deeper extraction.

Phase 3: Retrieval and evidence selection

1. **Hybrid retrieval:** for each slot, execute the dynamic queries over the evidence store. Use BM25 and dense embeddings to retrieve text, table and figure candidates. Evaluate both text and tables and use GROBID labels to prioritise sections likely to contain design details (e.g., “Design and Implementation”).
2. **Visual retrieval:** to capture diagrams that describe geometry, apply patch-based visual retrieval models such as **ColPali** or **ColQwen**. They embed page images and search using text queries (e.g., “horn aperture geometry” or “feed network”).
3. **Iterative retrieval:** if initial queries yield insufficient evidence, call the LLM again to propose additional synonyms or to refine search terms based on partial results. This dynamic loop prevents static retrieval from missing critical evidence.
4. **Reranking and diversity:** use a cross-encoder reranker such as **BGE-reranker v2** to score query-passage pairs; combine this with maximal marginal relevance to select a diverse set of evidence across sections, tables and figures.
5. **Per-slot evidence caps:** apply per-slot caps (e.g., top 3 items for parameters, 2 for materials) similar to the current pipeline but adjusted dynamically. Avoid the fixed caps of 2 or 4 used currently ⁸ when the design is more complex.
6. **Retrieval summary:** save a summary including the queries executed, number of candidates, evidence IDs and missing fields. This summary is part of the structured outputs for early review.

Phase 4: Extraction and claim generation

1. **Structured extraction from text and tables:**
2. Use **GROBID**’s full text to identify parameter tables, lists and inline measurements. Extract numeric values and units using deterministic regex heuristics.
3. Use **NuExtract3** to extract complex numeric data or labels from tables and figures. NuExtract3 accepts a JSON template describing the fields to extract; the output JSON will have the same structure, with missing values represented as `null` or empty arrays ⁹ ¹⁰. Example template for a horn antenna:

```
{
  "aperture_width": "number",
  "aperture_height": "number",
  "flare_angle": "number",
  "throat_width": "number",
  "throat_height": "number",
  "horn_length": "number"
}
```

4. NuExtract3 supports multimodal inputs (text, images or both) and uses reasoning or non-reasoning modes ¹¹. Use reasoning mode when the figure context needs interpretation.
5. **Visual extraction:** for images where geometry is purely visual, crop the region around the antenna diagram and pass it to NuExtract3 with a specific template (e.g., ask for the shape of the radiating element or feed location). If NuExtract3 fails, fall back to **Qwen2.5-VL** or **MonkeyOCR** to extract the shapes and positions.
6. **Claim store:** convert each extracted fact into a **Claim** record:

```
{
  "claim_id": "c42",
  "slot": "aperture_width",
  "value": 56.2,
  "unit": "mm",
  "evidence_ids": ["table_row:table_05:r03"],
  "confidence": 0.92
}
```

Claims are keyed by slot name and include the evidence IDs and a confidence score. Store all claims, including conflicts or ambiguous values.

Phase 5: Canonicalisation

1. **Canonical design selection:** run an open-source LLM (Gemma 4 or LLaMA-3) on the set of claims and the interpretation record to determine the final design variant. The prompt must be neutral regarding antenna types. A new system message might read:

You are the canonicalisation layer for antenna extraction. You are given claims about radiating elements, geometry, materials, feeds, layers and performance metrics for a single antenna design. Identify the final design variant described in the paper and resolve conflicts. Represent all radiating elements (patches, horns, helices, loops, reflectors, metamaterials, etc.) without bias. Do not invent missing parameters and preserve unresolved ambiguities.

1. **CanonicalDesignRecord:** the model outputs a record with fields: `antenna_class`, `radiating_elements` (list of structures with shape and dimensions), `materials`, `layers`,

`feeds`, `performance`, `ambiguous_facts`, `missing_required_for_build`. Each field contains lists of claims grouped by type.

Phase 6: Schema construction and final outputs

1. **GenericAntennaDesign schema:** design a flexible JSON schema that can represent arbitrary antennas. Key fields include:
 2. `antenna_class`: string enumeration (patch, array, horn, helix, slot, reflector, metamaterial, other).
 3. `radiating_elements`: list of objects containing `element_id`, `shape`, `dimensions` (dictionary of named dimensions and values), `position` (coordinates or relative description) and `evidence_ids`.
 4. `layers`: list of layers with `material`, `thickness`, `relative_permittivity` and `evidence_ids`.
 5. `materials`: list of materials used, with assignments to specific elements or layers.
 6. `feeds`: list of feed structures with type, shape, location and matching claims.
 7. `performance`: list of operating frequencies, bandwidths, gains, directivity and other metrics.
 8. `ambiguities` and `missing_fields`: capture unresolved conflicts or missing essential information.
 9. `evidence_used`: global list of evidence IDs referenced in the final record.
10. **Assembly:** write a deterministic function to map the `CanonicalDesignRecord` into the `GenericAntennaDesign`. The function chooses the correct enumeration values, populates dimension dictionaries, normalises units, and carries over evidence IDs. If a field cannot be represented, it is recorded in `ambiguities` or `missing_fields` rather than dropped.
11. **Structured outputs:** deliver the following human-readable JSON files:
 12. `interpretation_record.json`: classification, dynamic queries and open questions.
 13. `retrieval_summary.json`: queries run, candidates retrieved, and missing slots.
 14. `claims.json`: all extracted claims with confidence scores.
 15. `canonical_design_record.json`: canonicalised design with ambiguity notes.
 16. `generic_antenna_design.json`: final schema with antenna characteristics.

This separation avoids large, noisy intermediate JSONs and provides clarity at each stage.

3 Model recommendations

Pipeline stage	Recommended model(s)	Rationale and citations
Parsing and section structuring	GROBID	Extracts and structures PDFs into TEI/XML; provides full text segmentation and coordinates for figures, tables and references ³ .
Primary layout parser	Docling	Handles messy documents and creates page objects; used to build the evidence store.
Fallback OCR & layout	MonkeyOCR v1.5, olmOCR-2	Recognise multi-level tables, formulas and complex layouts; good fallback when Docling or GROBID fail.
Visual Markdown conversion and structured extraction	NuExtract3	Combines structured extraction and image-to-Markdown conversion; accepts text/images + JSON template and outputs JSON ¹¹ ; uses multimodal inputs and supports reasoning/non-reasoning modes ¹¹ ; returns <code>null</code> for missing fields and supports a variety of template types ¹⁰ .
Visual figure analysis	Qwen2.5-VL, ColPali/ ColQwen	Qwen2.5-VL handles diagrams, charts and tables; ColPali/ ColQwen perform visual retrieval to find relevant pages when text retrieval fails.
Embedding and search	BGE-m3	Multilingual dense embeddings for candidate generation.
Reranking	BGE-reranker v2	Cross-encoder reranker for query–passage scoring; used after BM25 + dense retrieval.
Antenna classification and reasoning	Gemma 4 or LLaMA-3	Open-weight LLMs with long context windows (Gemma supports up to 256K tokens) and reasoning ability; used for classification, query planning, canonicalisation and conflict resolution.

4 Conclusion

A generic antenna extraction pipeline must be adaptive rather than deterministic. The current **antennaAuto** design uses static query lists and emphasises patch geometry ¹ ² . By combining GROBID and Docling for parsing ³ , using NuExtract3 for precise structured extraction ¹¹ , generating dynamic queries based on LLM-driven classification, and employing flexible schemas, we can support arbitrary antennas, including horns, helices, arrays and fractal designs. Intermediate outputs such as interpretation records, retrieval summaries and claim stores provide transparency and ensure that human reviewers can audit the process without wading through noisy JSON. This architecture leverages open-source models to achieve high recall and precision while preserving the extensibility needed for future antenna types.

1 8 **raw.githubusercontent.com**

https://raw.githubusercontent.com/GoncaGomes/antennaAuto/326fe8dab0a761773bdf74327725da8afe59e7f5/docs/current_extraction_pipeline_detailed_qa.md

2 **raw.githubusercontent.com**

https://raw.githubusercontent.com/GoncaGomes/antennaAuto/326fe8dab0a761773bdf74327725da8afe59e7f5/src/mvp/prompts/canonicalization_system.md

3 4 5 **Introduction - GROBID Documentation**

<https://grobid.readthedocs.io/en/latest/Introduction/>

6 7 9 10 11 **numind/NuExtract3 · Hugging Face**

<https://huggingface.co/numind/NuExtract3>